Bayesian Personalized Ranking for Anime Recommendation

A reproducible implicit-feedback pipeline with point-in-time features and an honest comparison against tuned neighbourhood baselines

Patrick Oliveira

October 15, 2026

Abstract

We implement Bayesian Personalized Ranking (BPR) matrix factorization for top-N anime recommendation from implicit feedback, and benchmark it against most-popular, item-kNN and random baselines under a leave-last-one-out protocol. The contribution of this report is methodological rather than algorithmic: every aggregate feature is computed *point-in-time*, all hyperparameters are selected on a validation partition that is disjoint from the test partition, and the served ranking is asserted to be identical to the ranking that was scored offline. Our headline result is negative for the flagship model: a properly tuned item-kNN attains NDCG@10 of 0.1305 versus 0.1248 for tuned BPR. We report this rather than tuning until the preferred method wins, and we analyse the data regime that explains it.

Contents

1 Notation **3**

2 Introduction **3**
2.1 Scope and claims . . . 3
2.2 Why the negative result is kept . . . 3
2.3 Repository . . . 4

3 Data **4**
3.1 Source . . . 4
3.2 From ratings to implicit feedback . . . 4
3.3 Core filtering . . . 4
3.4 Temporal ordering and its limitation . . . 4

4 Feature engineering **5**
4.1 Rationale . . . 5
4.2 Point-in-time computation . . . 5
4.3 Fitting discipline . . . 6
4.4 Leakage controls . . . 6
4.5 A dtype bug found by these tests . . . 6

5 Methodology **7**
5.1 The BPR criterion . . . 7
5.2 Matrix factorization scorer . . . 7
5.3 Gradients . . . 7

5.4 Vectorization and why it changed the conclusions . . . 8
5.5 Negative sampling . . . 8
5.6 Baselines . . . 8

6 Evaluation protocol **8**
6.1 Partitioning . . . 8
6.2 Candidate set . . . 9
6.3 Metrics . . . 9
6.4 Model selection . . . 9
6.5 Training/serving parity . . . 10

7 Results **10**
7.1 Reading the table . . . 10
7.2 Hyperparameter search . . . 10
7.3 Why the baseline wins here . . . 10
7.4 Coverage . . . 11
7.5 Threats to validity . . . 11

8 Conclusion **11**
8.1 What transfers . . . 11
8.2 Future work . . . 12

1 Notation

Let $\mathcal{U}$ be the set of users and $\mathcal{I}$ the set of items, with $|\mathcal{U}| = m$ and $|\mathcal{I}| = n$. Implicit feedback is the set of observed positive interactions

$$\mathcal{S} \subseteq \mathcal{U} \times \mathcal{I}, \tag{1}$$

and $\mathcal{I}_u^+ = \{\, i \in \mathcal{I} : (u, i) \in \mathcal{S} \,\}$ denotes the items consumed by user u. A model produces a score $\hat{x}_{ui} \in \mathbb{R}$ used only for ranking; its absolute value carries no meaning.

Latent factors are written $\mathbf{p}_u \in \mathbb{R}^d$ for users and $\mathbf{q}_i \in \mathbb{R}^d$ for items, collected in matrices $P \in \mathbb{R}^{m \times d}$ and $Q \in \mathbb{R}^{n \times d}$. The full parameter set is $\Theta = \{P, Q, \mathbf{b}\}$ with item biases $\mathbf{b} \in \mathbb{R}^n$. The logistic function is $\sigma(z) = (1 + e^{-z})^{-1}$.

2 Introduction

Recommendation from implicit feedback differs from rating prediction in a way that is easy to state and easy to get wrong. We never observe a negative signal. A title a user has not watched may be a title they would dislike, or simply one they have not encountered. Optimizing a squared error against a zero-filled matrix therefore encodes an assumption that is false by construction: that absence of interaction means dislike.

Bayesian Personalized Ranking [3] replaces that assumption with a weaker and more defensible one. Rather than predicting an absolute score, it assumes only that an observed item is preferred to an unobserved one, and optimizes the probability of that *pairwise* ordering. The learning signal is a triplet (u, i, j) where i was consumed and j was not, and the objective rewards $\hat{x}_{ui} > \hat{x}_{uj}$.

2.1 Scope and claims

This report accompanies a working repository rather than a novel method. The claims we make are:

1. The pipeline is leakage-free. Every aggregate feature is computed point-in-time, and this is enforced by tests that inject a future event and assert no feature changes (Section 4).
2. The reported numbers are honest. Hyperparameters for *every* model, baselines included, were selected on a validation partition; the test partition was read once (Section 6).
3. The served ranking equals the evaluated ranking. Training/serving skew is asserted away rather than assumed away (Section 6).

We explicitly do *not* claim that BPR is the best model for this dataset. Section 7 shows it is not.

2.2 Why the negative result is kept

Dacrema et al. [1] reproduced eighteen neural recommendation papers and found that most were beaten by well-tuned non-neural baselines, largely because the baselines in the original papers had not been tuned at all. The cheapest way to produce a flattering result for a flagship model is to compare it against an untuned competitor. We tune the baselines on the same grid budget as the flagship model, and report the outcome as measured.

2.3 Repository

The implementation, infrastructure, and the scripts that generate every table in this document are available in the accompanying repository. The results table (Table 2) is emitted by `scripts/export_results_table.py` and the feature catalogue (Table 1) by `scripts/export_feature_tables` so the document cannot drift away from the code.

3 Data

3.1 Source

The dataset is the Kaggle *Anime Recommendations Database*, containing approximately 73,000 users, 12,000 titles and 7.8 million ratings. Each row is a triple `(user_id, anime_id, rating)` where the rating is an integer in $[1, 10]$, or the sentinel -1 meaning the user watched the title but did not rate it.

The dataset is not redistributed with the repository. A generator produces a synthetic fixture with the same schema, latent taste clusters and a Pareto popularity distribution, so that the full pipeline and its tests run without credentials. All numbers in this report are computed on the fixture; the qualitative conclusions are what transfer, not the absolute values.

3.2 From ratings to implicit feedback

We map explicit ratings to implicit positives:

$$(u, i) \in \mathcal{S} \iff r_{ui} = -1 \;\text{ or }\; r_{ui} \geq 6. \tag{2}$$

Two decisions in Equation 2 deserve justification.

The sentinel is a positive. A user who watched a title without rating it has revealed consumption, which is exactly the signal BPR models. Discarding these rows would throw away the majority of the interaction data.

Low ratings are dropped, not treated as negatives. A rating of 2 is evidence of *dislike*, which is strictly more information than the "unobserved" state BPR assumes for the sampled item j. Placing a known-disliked item into the negative pool would be defensible; placing it into the positive pool would not. We drop these rows because the sampler draws negatives from the complement of $\mathcal{I}_u^+$, and a disliked item correctly belongs there by default.

3.3 Core filtering

Users and items with very few interactions carry almost no collaborative signal and inflate the catalogue over which ranking metrics are computed. We apply a k-core filter with $k = 5$ on both sides.

The filter is applied *iteratively*. Removing low-activity items reduces user interaction counts, which can push a previously qualifying user below the threshold; a single pass would leave the stated invariant violated. The loop runs until a fixed point, so the output genuinely satisfies

$$|\mathcal{I}_u^+| \geq k \;\; \forall u \in \mathcal{U} \qquad \text{and} \qquad |\mathcal{U}_i^+| \geq k \;\; \forall i \in \mathcal{I}. \tag{3}$$

3.4 Temporal ordering and its limitation

The raw dataset carries **no timestamps**. This is a real constraint and we state it plainly rather than inventing one. Point-in-time feature computation requires an ordering, so we derive a per-user sequence index

Table 1: Engineered features. Every aggregate is computed point-in-time: only interactions strictly before the prediction cut-off contribute to its value.

Feature	Family	Transform	Point-in-time
`user_activity_count`	interaction	`user_activity_count`	✓
`user_rating_mean`	interaction	`user_rating_mean`	✓
`item_popularity_rank`	item	`item_popularity_rank`	✓
`item_member_count`	item	`item_member_count`	–
`item_genre_tokens`	text	`item_genre_tokens`	–
`user_genre_affinity`	interaction	`user_genre_affinity`	✓

$$t_{ui} = \text{position of } i \text{ within the rows of } u \quad \text{(file order)}, \tag{4}$$

and treat it as a weak proxy for chronology.

This proxy is adequate for *enforcing* the leakage discipline — the machinery that forbids a feature from reading beyond a cut-off is exercised and tested regardless of whether the cut-off is a true timestamp. It is *not* adequate for claiming that the evaluation reproduces a real production split. On a dataset with genuine timestamps, the same code path takes the timestamp column instead, and the synthetic fixture carries real timestamps precisely so that the point-in-time tests exercise that path.

4 Feature engineering

Features are declared in `conf/features.yaml` and executed by a registry-backed pipeline. The YAML is the single source of truth: a startup check validates the mapping between declared features and implemented transforms in *both* directions, so a feature cannot be declared without an implementation, nor can an implementation silently linger after its declaration is removed.

4.1 Rationale

`user_activity_count` Separates heavy watchers from casual users. Drives confidence weighting and explains why sparse users need the popularity fallback.

`user_rating_mean` Captures user leniency so a score of 7 is read relative to the user's own scale rather than an absolute threshold.

`item_popularity_rank` Popularity prior used by the most-popular baseline and as the cold-start fallback ordering for users with no training history.

`item_member_count` Catalogue-side audience size. Known before any interaction exists, so it is safe for cold-start items and is not subject to the train-only rule.

`item_genre_tokens` Content signal that allows unseen titles to be scored by genre overlap instead of being excluded from recommendation entirely.

`user_genre_affinity` Time-decayed taste profile over genres. Recent watches matter more than old ones, so interest fades rather than accumulating forever.

4.2 Point-in-time computation

Every aggregate is evaluated against a cut-off τ using a *strict* inequality:

$$\phi(u, \tau) = f\big(\{\, (u, i, t) \in \mathcal{S} : t < \tau \,\}\big). \tag{5}$$

The strictness matters. With $t \leq \tau$, the interaction being predicted contributes to its own feature value, and the model receives a trace of the label. This is the most common leakage bug in recommender pipelines because it produces excellent offline metrics and no error message.

Concretely, user activity count and the rating mean are

$$\text{activity}(u, \tau) = \big|\{\, (u, i, t) \in \mathcal{S} : t < \tau \,\}\big|, \tag{6}$$

$$\bar{r}(u, \tau) = \frac{1}{\text{activity}(u, \tau)} \sum_{(u,i,t) \in \mathcal{S},\, t<\tau} r_{ui}, \tag{7}$$

with $\bar{r}(u, \tau)$ defined as a configured fill value when the denominator is zero.

Genre affinity is a time-decayed profile over the genre vocabulary G. For genre g,

$$\text{aff}(u, g, \tau) = \frac{\sum_{(u,i,t) \in \mathcal{S},\, t<\tau} \lambda^{\tau - t} \mathbb{1}[g \in G_i]}{\sum_{(u,i,t) \in \mathcal{S},\, t<\tau} \lambda^{\tau - t}}, \qquad \lambda \in (0, 1], \tag{8}$$

so that recent consumption dominates a user's represented taste.

Item popularity is a rank, not a raw count:

$$\text{poprank}(i, \tau) = \big|\{\, i' \in \mathcal{I} : c_{i'}(\tau) > c_i(\tau) \,\}\big| + 1, \qquad c_i(\tau) = \big|\{\, (u, i, t) \in \mathcal{S} : t < \tau \,\}\big|. \tag{9}$$

Ranking is deliberate: raw counts follow a Pareto distribution whose scale shifts as the catalogue grows, whereas the rank is stable under that growth.

4.3 Fitting discipline

Statistics that require fitting — popularity tables, encodings, normalizer parameters — are fitted on the *training partition only* and then applied to validation and test. Fitting on the full dataset is leakage even when the per-row computation respects the cut-off, because the fitted parameters summarise the held-out rows.

4.4 Leakage controls

Three classes of test enforce the above, in `tests/test_features_leakage.py`:

1. **Future-event injection.** An interaction is appended *after* the cut-off and every feature is recomputed. Any change in any value fails the test. This is the direct executable form of Equation 5.
2. **Fit-partition isolation.** Fitted tables computed on the training partition are asserted identical whether or not the validation and test rows are present in the input.
3. **Label destruction.** Item assignments in the training partition are randomly permuted, destroying the user–item association while preserving every marginal distribution. A model retrained on the corrupted data must collapse towards the random baseline. If it does not, some feature is carrying the answer.

The third test is the one that catches leakage the structural checks miss, because it makes no assumption about *which* feature is responsible.

4.5 A dtype bug found by these tests

The future-event injection test initially failed with an index dtype mismatch (`int32` versus `int64`) rather than a value difference. The cause was that `pandas` promotes index dtypes differently depending on how a frame was assembled, so two logically identical feature frames compared unequal. The fix normalises every user-keyed series to an `int64` index at construction. We record it here because it is a genuine robustness defect that would have surfaced as a confusing production failure, not a test artifact that was patched away.

5 Methodology

5.1 The BPR criterion

BPR starts from a per-user total order $>_u$ over items and assumes only that observed items are preferred to unobserved ones. The training set of triplets is

$$D_{\mathcal{S}} = \{\, (u, i, j) \mid i \in \mathcal{I}_u^+,\ j \in \mathcal{I} \setminus \mathcal{I}_u^+ \,\}. \tag{10}$$

Assuming independence across users and triplets, the posterior over parameters given the orderings is

$$p(\Theta \mid >_u) \propto p(>_u \mid \Theta)\, p(\Theta), \tag{11}$$

and with the pairwise likelihood modelled as $p(i >_u j \mid \Theta) = \sigma(\hat{x}_{uij})$ and a Gaussian prior $\Theta \sim \mathcal{N}(0, \lambda^{-1} I)$, maximizing the posterior is equivalent to maximizing

$$\text{BPR-Opt} = \sum_{(u,i,j) \in D_{\mathcal{S}}} \ln \sigma(\hat{x}_{uij}) \;-\; \lambda \|\Theta\|^2, \tag{12}$$

where the pairwise score difference is

$$\hat{x}_{uij} = \hat{x}_{ui} - \hat{x}_{uj}. \tag{13}$$

5.2 Matrix factorization scorer

We instantiate the scorer as biased matrix factorization:

$$\hat{x}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i + b_i = \sum_{f=1}^{d} p_{uf} q_{if} + b_i. \tag{14}$$

The user bias is deliberately absent. A per-user constant shifts every item score for that user by the same amount and therefore cannot change the induced ranking; including it would only add an unidentifiable parameter. The item bias survives because it *does* reorder items, acting as a learned popularity prior. Note also that b_i cancels from $\hat{x}_{uij}$ only when $i = j$, which never occurs.

5.3 Gradients

Write $\delta = \sigma(-\hat{x}_{uij})$, which follows from $\frac{\partial}{\partial z} \ln \sigma(z) = \sigma(-z)$. For a single triplet the gradients of Equation 12 are

$$\frac{\partial}{\partial \mathbf{p}_u} = \delta\, (\mathbf{q}_i - \mathbf{q}_j) - \lambda_u \mathbf{p}_u, \tag{15}$$

$$\frac{\partial}{\partial \mathbf{q}_i} = \delta\, \mathbf{p}_u - \lambda_i \mathbf{q}_i, \tag{16}$$

$$\frac{\partial}{\partial \mathbf{q}_j} = -\delta\, \mathbf{p}_u - \lambda_i \mathbf{q}_j, \tag{17}$$

$$\frac{\partial}{\partial b_i} = \delta - \lambda_b b_i, \tag{18}$$

$$\frac{\partial}{\partial b_j} = -\delta - \lambda_b b_j. \tag{19}$$

The factor δ has a useful reading: it is large when the model currently ranks the triplet *wrongly* and tends to zero once the pair is confidently ordered, so converged triplets stop contributing gradient.

Parameters are updated by ascent with step size η decayed geometrically per epoch, $\eta_e = \eta_0 \gamma^e$.

5.4 Vectorization and why it changed the conclusions

A literal reading of Equation 19 gives one update per triplet. In NumPy that costs roughly 70 seconds for 40 epochs on the fixture, which makes a hyperparameter grid search impractical — and an untuned flagship model compared against baselines would have reproduced exactly the methodological failure criticised in Section 2.

We therefore apply the same gradients to mini-batches of B triplets. One subtlety makes this non-trivial: a user or item may appear several times within a batch, and NumPy's fancy-indexed assignment `A[idx] += v` keeps only the *last* write for duplicated indices, silently discarding gradients. We use `np.add.at`, which accumulates. The resulting implementation runs the same 40 epochs in 2.3 seconds, a $\sim 30\times$ speed-up, which is what made the grid search in Section 7 affordable.

5.5 Negative sampling

For each positive (u, i) the sampler draws j uniformly from $\mathcal{I} \setminus \mathcal{I}_u^+$. Rejection sampling is the natural approach, but testing membership against a per-user Python set inside the training loop reintroduces the interpreter overhead that vectorization removed, and a dense $m \times n$ boolean mask would need roughly 876 MB on the full dataset.

We instead encode each observed pair as a single integer key

$$\kappa(u, i) = u \cdot n + i, \tag{20}$$

store the sorted key array, and test candidate membership with a vectorized binary search. Memory is $O(|\mathcal{S}|)$ rather than $O(mn)$, and collisions are resampled for a bounded number of rounds. A test asserts zero false negatives across 340,000 draws.

5.6 Baselines

Most-popular. Ranks all users by the same global interaction count. It is non-personalised by construction and serves as the floor that any personalised model must clear.

Item-kNN. Cosine similarity between binary item vectors,

$$s_{ij} = \frac{|\mathcal{U}_i^+ \cap \mathcal{U}_j^+|}{\sqrt{|\mathcal{U}_i^+|}\sqrt{|\mathcal{U}_j^+|}}, \tag{21}$$

truncated to the k nearest neighbours per item, with user score

$$\hat{x}_{ui} = \sum_{j \in \mathcal{I}_u^+} s_{ij}. \tag{22}$$

This is the method most Kaggle notebooks for this dataset actually use, so it is the bar that justifies the project.

Random. A seeded permutation per user. It calibrates the metric scale and, with full catalogue coverage, demonstrates that coverage alone is not a quality signal.

6 Evaluation protocol

6.1 Partitioning

We use leave-last-one-out per user. Ordering each user's interactions by the temporal proxy of Section 3, the final interaction forms the test instance, the penultimate one the validation instance, and everything earlier is training data.

Users with fewer than $k_{\min} + 2$ interactions cannot yield both held-out instances without starving their own training history; these users contribute entirely to the training partition and are not evaluated. Reporting metrics over users whose training history is empty would measure the popularity prior rather than personalisation.

6.2 Candidate set

For each evaluated user we rank the *entire* catalogue minus that user's training history:

$$C_u = \mathcal{I} \setminus \mathcal{I}_u^{+,\text{train}}. \tag{23}$$

Excluding the history is not a detail. Without it, a model is rewarded for re-recommending already-consumed titles, which inflates every metric and corresponds to a product behaviour nobody wants. A test constructs a model that returns only the user's history and asserts it scores exactly zero.

We rank the full catalogue rather than sampling 100 negatives per user. Sampled-negative evaluation is common but is known to produce rankings of methods that disagree with the full-catalogue result [2].

6.3 Metrics

With exactly one relevant item i_u^* per evaluated user and $\text{rank}_u \in \{1, \dots, |C_u|\}$ its position in the predicted list, the cut-off metrics over the evaluated user set $\mathcal{U}_{\text{eval}}$ are:

$$\text{NDCG@}K = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u} \frac{\mathbb{1}[\text{rank}_u \le K]}{\log_2(\text{rank}_u + 1)}, \tag{24}$$

$$\text{Recall@}K = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u} \mathbb{1}[\text{rank}_u \le K], \tag{25}$$

$$\text{MRR@}K = \frac{1}{|\mathcal{U}_{\text{eval}}|} \sum_{u} \frac{\mathbb{1}[\text{rank}_u \le K]}{\text{rank}_u}, \tag{26}$$

$$\text{Coverage@}K = \frac{1}{|\mathcal{I}|} \Big| \bigcup_{u \in \mathcal{U}_{\text{eval}}} \text{top}_K(u) \Big|. \tag{27}$$

Equation 24 uses the general NDCG definition specialised to a single relevant item, where the ideal DCG equals 1 and the normalisation therefore disappears. Under this protocol Recall@K coincides numerically with hit rate; both are reported so the table stays comparable with multi-positive projects.

Coverage is reported because it is the business reason to leave a popularity list, but it must be read alongside accuracy: the random baseline attains coverage 1.0 while being useless.

6.4 Model selection

All hyperparameters — for the baselines as well as for BPR — are selected by grid search on the *validation* partition. The test partition is read once, to produce Table 2.

This is stated explicitly because the alternative is both common and invisible in a results table. Selecting on test turns the reported number into an optimistically biased estimate whose gap from reality grows with the size of the grid.

Table 2: Top-K ranking quality on the held-out test partition ($K = 10$). Hyperparameters for every model were selected on the validation partition; the test partition was read once.

Model	NDCG@10	Recall@10	MRR@10	Coverage
ItemKNN	**0.1305**	0.2483	0.0952	0.4969
BPR	0.1248	0.2339	0.0922	0.3937
Popularity	0.0761	0.1448	0.0556	0.0750
Random	0.0150	0.0334	0.0095	1.0000

6.5 Training/serving parity

Offline metrics are worthless if production serves a different ranking. The serving path precomputes top-N lists into DynamoDB, and `tests/test_features_parity.py` asserts that the stored list for each user is element-wise identical to the list produced by the same fitted model during evaluation, that no served list intersects the user's history, and that the payload is JSON-serialisable — NumPy scalars are a frequent and late-surfacing cause of rejected writes.

7 Results

7.1 Reading the table

Both personalised models clear the most-popular baseline by a wide margin, and all three clear random by roughly an order of magnitude, so the pipeline is learning genuine collaborative structure.

The flagship model, however, loses. Tuned item-kNN reaches NDCG@10 of 0.1305 against 0.1248 for tuned BPR, a relative gap of about 4.6% in favour of the baseline. Recall@10 and MRR@10 order the two models the same way, so the result is not an artifact of a single metric's discount function.

7.2 Hyperparameter search

The BPR grid covered $d \in \{16, 32, 64, 128\}$, $\eta_0 \in \{0.02, 0.05, 0.1, 0.2\}$, $\lambda \in \{0.005, 0.02, 0.05, 0.1, 0.2\}$ and epoch budgets from 20 to 200, selected on validation NDCG@10. The optimum sits at $d = 32$, $\eta_0 = 0.2$, $\lambda = 0.05$.

Two features of the search are worth recording. First, validation performance is *non-monotonic* in the epoch budget: it peaks and then declines while the training objective keeps improving, which is overfitting of the pairwise criterion rather than of a reconstruction error. Second, performance is far more sensitive to λ than to d — the swing across the regularization axis is roughly 0.05 NDCG, against roughly 0.005 across the factor axis. The item-kNN neighbourhood size was swept over $k \in \{10, 20, 50, 100, 200, |\mathcal{I}|\}$ and is remarkably flat, varying by under 0.004 NDCG across the entire range.

That flatness is itself informative: the baseline is near-optimal almost regardless of its one hyperparameter, while BPR requires a search to become competitive at all.

7.3 Why the baseline wins here

The result is consistent with the data regime rather than with a defect in the implementation, and three observations support that reading.

Scale. Matrix factorization amortises a shared low-rank structure across many observations. The evaluated fixture has $\sim$ 900 users and 320 items. Repeating the comparison at roughly

five times the scale ($\sim$ 5,000 users, 1,500 items, 95,000 training interactions) did not reverse the ordering, but it did compress the gap, which is the direction the scale argument predicts.

Density. The k-core filter leaves a comparatively dense interaction matrix. Co-occurrence counts are therefore well estimated directly, and the cosine similarities that item-kNN consumes are low variance. Neighbourhood methods are strongest exactly in this regime.

Intrinsic rank. The latent structure is a small number of taste clusters crossed with a popularity prior. Item-kNN represents within-cluster co-consumption essentially exactly, whereas BPR must recover the same structure through a rank-d approximation fitted by sampled SGD. There is little headroom above a model that already captures the generating process.

7.4 Coverage

BPR covers 39% of the catalogue against 50% for item-kNN and 7.5% for most-popular. The popularity baseline's figure quantifies the concentration problem that motivates personalisation: it is structurally incapable of surfacing more than a handful of titles. Random attains 100% coverage with near-zero accuracy, which is the reminder that coverage is a constraint to satisfy, not an objective to maximise.

7.5 Threats to validity

1. **Synthetic fixture.** The reported numbers come from generated data. The generating process was not chosen to favour either model, but it is simpler than reality, and simple structure favours the method with the stronger inductive bias.
2. **Proxy ordering.** The split uses file order as a chronology proxy (Section 3). A true temporal split would be harder, and would likely lower all reported numbers.
3. **Single seed.** Metrics are reported for one seed. The gap between BPR and item-kNN is small enough that a confidence interval over seeds would be needed before calling it decisive.
4. **Offline only.** NDCG on held-out interactions is a proxy for user satisfaction and cannot capture novelty, diversity or presentation effects.

8 Conclusion

We implemented BPR matrix factorization end to end: implicit-feedback construction, iterative core filtering, point-in-time feature engineering, vectorized triplet SGD, full-catalogue ranking evaluation, and a parity-tested serving path deployed by Terraform against a local AWS emulator.

The measured outcome is that a tuned item-kNN baseline edges out tuned BPR on this data (NDCG@10 0.1305 versus 0.1248). We report it because the alternative — tuning until the flagship model wins — is the specific failure mode that Dacrema et al. [1] documented across a large body of published recommender work, and avoiding it was an explicit goal of this project.

8.1 What transfers

The reusable artifacts of this project are not the model weights:

- **A leakage harness.** Future-event injection, fit-partition isolation, and label destruction, as executable tests rather than prose. The injection test surfaced a genuine dtype-promotion defect in the feature layer.

- **A parity harness.** Asserting that the served ranking equals the evaluated ranking closes the gap where good offline numbers quietly fail to reach production.
- **An honest selection protocol.** Validation-only tuning, with the same grid budget extended to the baselines.

8.2 Future work

Popularity-weighted sampling. Drawing negatives j with probability proportional to c_j^α rather than uniformly produces harder triplets as training progresses, and is the most likely single change to close the observed gap.

Confidence intervals. Repeating the comparison across seeds and reporting intervals would establish whether the 0.006 NDCG gap is significant.

Hybrid scoring. The content features in Table 1 are currently consumed only for cold-start fallback. Folding them into the scorer, $\hat{x}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i + b_i + \mathbf{w}^\top \boldsymbol{\phi}_{ui}$, would let the model exploit genre affinity directly.

Real timestamps. Evaluating on a dataset with genuine event times would replace the chronology proxy and make the temporal split defensible as a simulation of production.

References

[1] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? a worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*, pages 101–109, 2019.

[2] Walid Krichene and Steffen Rendle. On sampled metrics for item recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1748–1757, 2020.

[3] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 452–461, 2009.